

EMPLOYEE PAYROLL SYSTEM

BUSINESS CASE STUDY

A Python-based solution for employee records, salary calculation, payslip generation and JSON data storage.

PROJECT OBJECTIVE

Build a simple menu-driven payroll application that manages employee information, calculates gross and net salary, generates payslips, searches employee records and stores payroll data persistently using JSON.

Prepared by: Shriya Verma

Aspiring Data Analyst

01 BUSINESS PROBLEM

Small organizations may need a simple way to maintain employee salary records and perform routine payroll calculations. Manual handling can involve repeated data entry, calculation work and difficulty retrieving individual employee information.

02 PROPOSED SOLUTION

The Employee Payroll System centralizes the basic workflow in a Python application. It stores employee records in a list of dictionaries, uses JSON for persistent storage, and provides a menu-driven interface for common payroll operations.

03 KEY FEATURES

Add Employee: Store Employee ID, name, department, basic salary, allowances and deductions.

Calculate Salary: Compute gross and net salary.

Generate Payslip: Display a structured salary breakdown.

Search Employee: Retrieve records using Employee ID.

Update Employee: Correct an employee's name using Employee ID.

Save Payroll Data: Persist records in `payroll.json`.

04 PAYROLL CALCULATION LOGIC

Gross Salary = Basic Salary + Allowances

Net Salary = Gross Salary – Deductions

Example: Basic Salary ■25,000 + Allowances ■4,500 = Gross Salary ■29,500. After ■2,000 deductions, Net Salary = ■27,500.

05 DATA STORAGE

The project uses Python's built-in JSON module. Existing records are loaded from **payroll.json** when available, and updated records are written back to the same file. This provides simple persistence without requiring a database.

06 SYSTEM WORKFLOW

Start → Load payroll.json → Display Menu → Choose Operation → Add / Calculate / Payslip / Search → Update Records → Save Data → Continue or Exit

07 BUSINESS VALUE

The project demonstrates how a repetitive administrative process can be structured into reusable software functions. It supports consistent salary calculations, faster employee lookup, formatted payslip generation and persistent record storage.

08 TECHNICAL SKILLS

Python functions, lists, dictionaries, loops, conditional statements, user input, file handling, JSON serialization and deserialization, formatted console output, record searching, record updating and menu-driven application design.

09 PROJECT LIMITATIONS

This is a learning project rather than a production payroll platform. It uses JSON instead of a relational database and does not implement full tax, attendance, statutory deduction, authentication or enterprise payroll workflows.

10 FUTURE ENHANCEMENTS

Potential improvements include complete employee editing and deletion, stronger input validation, tax and PF calculations, attendance-based salary processing, monthly payroll reports, PDF payslip export, a graphical user interface and MySQL/database integration.

11 CONCLUSION

The Employee Payroll System connects Python programming concepts with a practical business process. It demonstrates how structured employee data, payroll formulas, search functionality, formatted outputs and persistent storage can be combined into a small functional business application.